ITDBADM

Quiz 1 Reviewer – February 3

Reminders

1. DB App Proposal Draft in the google sheets: https://docs.google.com/spreadsheets/d/1-S2IsL_fLRZcScZ1qUWNpRbIn6w27PwsDagXEhUTFyU/edit?gid=0#gid=0
2. Quiz 1 will be on February 6 (Friday)
   - Coverage:
     - SQL Tasks from Assignments and Seatwork
     - Creating Tables, Inner Joins, Aggregations
     - MCQ, Essay, and SQL Tasks
   - Closed notes. Quiz will be uploaded in Canvas using the Desktop Computer in GK304B. Laptops not allowed for the quiz.

Final Project Specifications

- Create a database for a “unique” store with **societal impact**.
- **GUI FRONT-END (JAVA, PYTHON, PHP, HTML Any is okay)**
  - Login (based on roles) (At least 3 Roles: Admin, Staff, Customer). Access to specific tables should be controlled.
  - Browse products/ Services (At least 6 product types and/or service types)
  - Currency handling (At least 3 currencies. E.g. PHP, US Dollar, Korean Won, etc.)
  - The store should at least have 2 branches.
  - Admin panel (Add/edit products and services)
  - Add to cart and place an order. View order history.
  - Log Tables when using Triggers.
  - Should be directly integrated with MySQL Workbench only! (Do not use MongoDB, PostgreSQL, phpMyAdmin)
- **DELIVERABLES**
  1. **SQL FILES:** Schema, Stored Procedures, Triggers, etc.
     - Should include 6 tables. Justify the relationships of the tables.
     - Should have at least 10 procedures and 10 triggers
  2. **GUI Source Codes** (Python, Java, PHP, etc.)
  3. **PPT PRESENTATION** (20 Minute Demo, 10 minute Q&A)
     - PPT Content: ONLINE STORE INTRO, ERD DIAGRAM, DESCRIPTION OF PROCEDURES & TRIGGERS, SCREENSHOTS OF GUI, SQL CODE SNIPPETS, EXAMPLE OUTPUTS

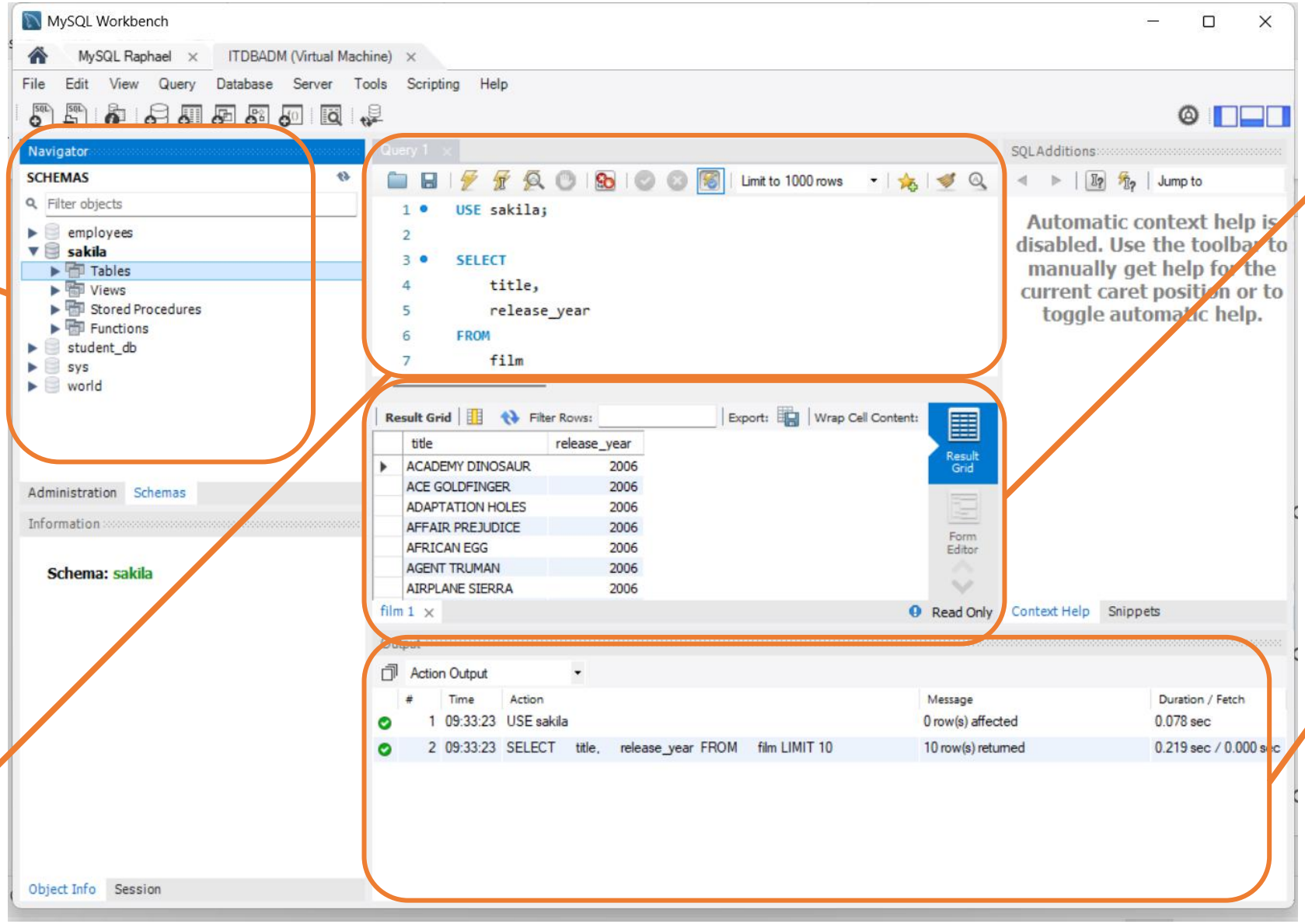
For today's session

- **Review of what we discussed so far.**
- **Quiz 1**
  - We will mainly use the SAKILA Database. Make sure you load the Sakila DB in the workbench in the computer lab.
  - The password in MySQL in the lab: **p@ssword**
  - No laptops. We will only use the computer in the Lab!
  - Closed notes. No cheating please! No kodigo. Do not hide sql statements in MySQL workbench.
  - The quiz will start from **9:45 am to 10:45 am (1 hour)**
  - Multiple Choice Questions
  - 1 Essay Question
  - SQL Tasks (Very similar to the assignments and lectures). Need to print screen your results.
  - Read the instructions carefully!!!

MySQL Workbench Interface

Schemas Tab:
You can see your tables, views, and functions in your schemas

Query Editor Window:
This is where you write and run you code



Result Grid:
This shows the output of your SQL queries in a table format

Action Output:
A summary of actions taken by the server.

Note: The Message collum is useful for troubleshooting

Common MySQL Data Types

NUMERIC DATA TYPES	
Data Type	**Specifications**
TINYINT	Integer (-128 to 127)
SMALLINT	Integer (-32768 to 32767)
MEDIUMINT	Integer (-8388608 to 8388607)
INT	Integer (-2147483648 to 2147483647)
BIGINT	Integer (-9223372036854775808 to 9223372036854775807)
FLOAT	Decimal (precise to 23 digits)
DOUBLE	Decimal (23 to 53 digits)
DECIMAL	Decimal (to 65 digits – most precise)

STRING DATA TYPES	
Data Type	**Specifications**
CHAR	String (0 – 255)
VARCHAR	String (0 – 255)
TINYTEXT	String (0 – 255)
TEXT	String (0 – 65535)
DATE	YYYY-MM-DD
DATETIME	YYYY-MM-DD HH:MM:SS
TIMESTAMP	YYYYMMDDHHMMSS
ENUM	One of a number of preset options

What does a DBA do?

Focus	DBA	Data Analyst
Focus area	Keeps databases running smoothly, securely, and efficiently	Turns raw data into insights for effective business decision-making
Daily Work	Manages databases, enhances security, optimizes performance, and prevents data loss	Analyzes trends, build reports, and works with teams to improve business performance
Tools	Works with database systems like MySQL and cloud platforms to store and protect data	Power Bi, Tableau, SQL, Python, and Excel to analyze data
Business Impact	Assures that data is safe, reliable, and readily available so business can operate without disruption	Helps businesses spot opportunities, reduce risk, and grow through data-driven strategies

Basic Elements "Big 6" of a SQL SELECT Statement

Start

SELECT	**SELECT** columnName	***Identifies the column(s) you want your query to select for your results.***
FROM	**FROM** tableName	***Identifies the table(s) your query will pull data from***
WHERE	**WHERE** logicalCondition	***(Optional) Specifies record-filtering criteria for filtering your results***
GROUP BY	**GROUP** BY columnName	***(Optional) Specifies how to group the data in your results***
HAVING	**HAVING** logicalCondition	***(Optional) Specifies group-filtering criteria for filtering your results***
ORDER BY	**ORDER BY** columnName	***(Optional) Specifies the order in which your query results are displayed***

End

CREATE SCHEMA

- Used to create a new table in the database
- Syntax:

```
CREATE SCHEMA schema_name;
```

```
USE schema_name;
```

- Example:

```
CREATE SCHEMA pbb_collab;
USE pbb_collab;
```

```
CREATE SCHEMA sakila;
USE sakila;
```

CREATE TABLE

- Used to create a new table in the database
- Syntax:

```
CREATE TABLE table_name (
    column1 datatype,
    column2 datatype,    ...
);
```

```
-- Main table
CREATE TABLE students (
    student_id INT PRIMARY KEY AUTO_INCREMENT,
    name VARCHAR(100),
    gender ENUM('Male','Female','Other'),
    birth_date DATE,
    email VARCHAR(100)
);

-- Table connected to students
CREATE TABLE enrollments (
    enrollment_id INT PRIMARY KEY AUTO_INCREMENT,
    student_id INT,
    course_name VARCHAR(100),
    grade FLOAT,        -- e.g., 85.5
    credits INT,        -- e.g., 3
    enroll_date DATE,
    FOREIGN KEY (student_id) REFERENCES students(student_id)
);
```

INSERT INTO

- Used to insert new data into a table
- Syntax:

```
INSERT INTO table_name (column1, column2, ...)
VALUES (value1, value2, ...);
```

- Example:

```
INSERT INTO students (student_id, name, age)
VALUES (1, 'Alice', 21);
```

UPDATE

- Used to modify existing data in a table
- Syntax:

```
UPDATE table_name
SET column1 = value1, column2 = value2, ...
WHERE condition;
```

- Example:

```
UPDATE students
SET age = 22
WHERE student_id = 1;
```

DELETE

- Used to delete existing rows from a table
- Syntax:

```
DELETE FROM table_name
WHERE condition;
```

- Example:

```
DELETE FROM students
WHERE student_id = 1;
```

Use this to remove warning message → SET SQL_SAFE_UPDATES = 0;

ALTER TABLE

- Used to modify the structure of an existing table
- Common uses: Add a new column, Modify a column, Delete a column
- Example:

```
ALTER TABLE students ADD email VARCHAR(100);
ALTER TABLE students MODIFY age TINYINT;
ALTER TABLE students DROP COLUMN email;
```

LIMIT

- **LIMIT** is used to restrict the number of rows returned in the result set
- Common use: Preview only top results (e.g., top 5 customers)
- Example:

```
-- Get the first 5 rows from the customer table
SELECT * FROM customer
LIMIT 5;
```

DISTINCT

- **DISTINCT** is used to remove duplicate values from the result set
- List unique values from a column (e.g., unique cities)
- Example:

```
-- Get a list of unique first names
SELECT DISTINCT first_name FROM customer;
```

INNER JOIN

- Returns **only matching records** (e.g. customer_id) from both tables.
- Commonly used when you want data that exists **in both** tables.
- Example:

Table A: customer

customer_id	name
1	Ralph
2	Esnyr
3	Charlie

```
SELECT c.name, r.rental_date
FROM customer c
INNER JOIN rental r ON c.customer_id = r.customer_id;
```

Table B: rental

rental_id	customer_id	rental_date
101	1	2023-01-01
102	2	2023-01-02

Inner Joined Table

name	rental_date
Ralph	2023-01-01
Esnyr	2023-01-02

LEFT JOIN

- Returns **all records from the left table**, and matching records from the right table.
- If no match, NULLs are returned from the **right table**.

Table A: customer (LEFT TABLE)

customer_id	name
1	Ralph
2	Esnyr
3	Charlie

Table B: rental (RIGHT TABLE)

rental_id	customer_id	rental_date
101	1	2023-01-01
102	2	2023-01-02

```
SELECT c.name, r.rental_date
FROM customer c
LEFT JOIN rental r ON c.customer_id = r.customer_id;
```

Left Joined Table

name	rental_date
Ralph	2023-01-01
Esnyr	2023-01-02
Charlie	NULL

RIGHT JOIN

- Returns **all records from the right table**, and matching records from the left table.
- If no match, NULLs are returned from the **left table**.

Table A: customer (LEFT TABLE)

customer_id	name
1	Ralph
2	Esnyr

```
SELECT c.name, r.rental_date
FROM customer c
RIGHT JOIN rental r ON c.customer_id = r.customer_id;
```

Table B: rental (RIGHT TABLE)

rental_id	customer_id	rental_date
101	1	2023-01-01
102	2	2023-01-02
103	3	2023-01-03

Right Joined Table

name	rental_date
Ralph	2023-01-01
Esnyr	2023-01-02
NULL	2023-01-03

Summary Table

Statement / Keyword	Purpose
CREATE SCHEMA	Create a new schema or database
USE schema_name	Select a specific schema/database to work in
CREATE TABLE	Create a new table within a schema
INSERT INTO	Add new records to a table
UPDATE	Modify existing records in a table
DELETE	Remove records from a table
ALTER TABLE	Change the structure of an existing table (e.g., add/modify/drop columns)
LIMIT	Restrict the number of rows returned by a query
DISTINCT	Return only unique (non-duplicate) values
INNER JOIN	Return rows with matching values in both tables
LEFT JOIN	Return all rows from the left table, and matched rows from the right table (NULL if no match)
RIGHT JOIN	Return all rows from the right table, and matched rows from the left table (NULL if no match)

SQL Practice

- Task 1. Create a table named books with the following columns:

Column Name	Description
book_id	Primary Key, Number
title	Book title, String Type
author	Author name, String Type
price	Book price (e.g., 499.99)

```
CREATE TABLE books (
   book_id INT PRIMARY KEY,
   title VARCHAR(100),
   author VARCHAR(100),
   price DECIMAL(6,2)
);
```

- Task 2. Insert the following rows

book_id	title	author	price
1	The Great Gatsby	F. Scott Fitzgerald	399.99
2	To Kill a Mockingbird	Harper Lee	499.50

```
INSERT INTO books (book_id, title, author, price)
VALUES
(1, 'The Great Gatsby', 'F. Scott Fitzgerald', 399.99),
(2, 'To Kill a Mockingbird', 'Harper Lee', 499.50);
```

SQL Practice

- Task 3. Change the price of the Great Gatsby to 450.00

```
UPDATE books
SET price = 450.00
WHERE book_id = 1;
```

- Task 4. Delete the book “To Kill a Mockingbird”

```
DELETE FROM books
WHERE book_id = 2;
```

- Task 5. Add a new column named “genre” (type: VARCHAR(50)).

```
ALTER TABLE books
ADD genre VARCHAR(50);
```

Practice Prompts (Sakila Database)

Table: payment

payment_id	customer_id	staff_id	rental_id	amount	payment_date	last_update
1	1	1	76	2.99	2005-05-25 11:30:37	2006-02-15 22:12:30
2	1	1	573	0.99	2005-05-28 10:35:23	2006-02-15 22:12:30
3	1	1	1185	5.99	2005-06-15 00:54:12	2006-02-15 22:12:30
4	1	2	1422	0.99	2005-06-15 18:02:53	2006-02-15 22:12:30
5	1	2	1476	9.99	2005-06-15 21:08:46	2006-02-15 22:12:30
6	1	1	1725	4.99	2005-06-16 15:18:57	2006-02-15 22:12:30

AGGREGATE FUNCTIONS (MIN, MAX, COUNT, SUM, AVG)

MIN() → Finding the lowest number/smallest value
MAX() → Finding the highest number/largest vlaue

```
SELECT
   MIN(amount) AS lowest_profit,
   MAX(amount) AS highest_profit
FROM payment;
```

MIN MAX

lowest_profit	highest_profit
0.00	11.99

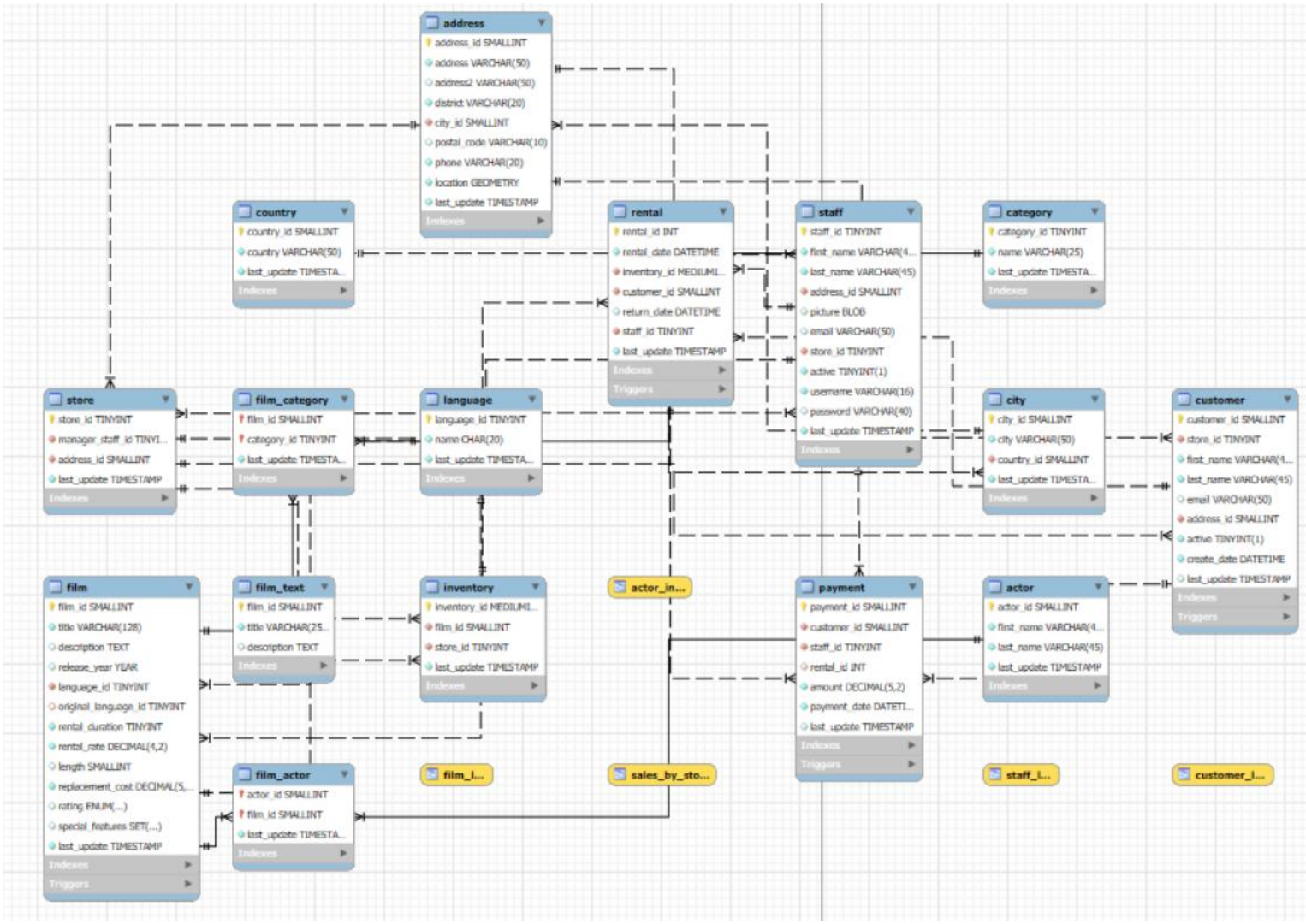
COUNT() → adding all the rows in a specific column
SUM() → adding all the values together
AVG() → calculate the average values

```
SELECT
   COUNT(amount) AS total_transactions,
   SUM(amount) AS total_profit,
   AVG(amount) AS average_profit
FROM payment;
```

COUNT SUM AVG

total_transactions	total_profit	average_profit
16044	67406.56	4.201356

Note: There are multiple ways to show the solutions. I will check your answers based on how you follow the instructions.
But it is recommended to just answer the task as simply and efficiently as possible.
You may include additional columns for "Verifiability" to make sure the requested data is correct.



Practice Prompts (Sakila Database)

AGGREGATE FUNCTIONS (MIN, MAX, COUNT, SUM, AVG)

Task 1: Count the Top 5 Customer Rentals
Tables Used: customer, rental
Expected Output Columns: customer_id, first_name, last_name, total_rentals
Statements Used: JOIN, COUNT(), ORDER BY, LIMIT

Task 2: Find the Highest Payment Made By Each Customer
Tables Used: payment, customer
Expected Output Columns: customer_id, first_name, last_name, max_payment
Statements Used: JOIN, MAX(), GROUP BY

Task 3: Show the duration (in days) for each rental, and sort to show the shortest ones first.
Tables Used: rental
Expected Output Columns: rental_id, rental_date, return_date, duration_days
Statements Used: DATEDIFF(), IS NOT NULL, ORDER BY

Task 4: Total Revenue per Store
Tables Used: payment, staff, store
Expected Output Columns: store_id, total_revenue
Statements Used: JOIN, SUM(), GROUP BY

Task 5: Average Payment Amount per Film
Tables Used: payment, rental, inventory, film
Expected Output Columns: film_id, title, average_payment
Statements Used: JOIN, AVG(), GROUP BY

Task 6: Find customers who have at least one rental that hasn't been returned yet. (Tip: return_date is null)
Tables Used: customer, rental
Expected Output Columns: customer_id, first_name, last_name, overdue_count
Statements Used: JOIN, IS NULL, COUNT(), GROUP BY, HAVING

Task 7: Display the total payment amount per month to identify revenue trends.
Tables Used: payment
Expected Output Columns: month_name, total_revenue
Statements Used: MONTH(), SUM(), GROUP BY

Task 1: Count the Top 5 Customer Rentals

```sql
SELECT
  c.customer_id,
  c.first_name,
  c.last_name,
  COUNT(r.rental_id) AS total_rentals
FROM customer AS c
JOIN rental AS r ON r.customer_id = c.customer_id
GROUP BY c.customer_id
ORDER BY total_rentals DESC
LIMIT 5;
```

customer_id	first_name	last_name	total_rentals
148	ELEANOR	HUNT	46
526	KARL	SEAL	45
236	MARCIA	DEAN	42
144	CLARA	SHAW	42
75	TAMMY	SANDERS	41

Task 2: Highest Payment Made By Each Customer

```
SELECT
  c.customer_id,
  c.last_name,
  c.first_name,
  MAX(p.amount) AS max_payment
FROM customer AS c
JOIN payment  AS p ON p.customer_id = c.customer_id
GROUP BY c.customer_id, c.first_name, c.last_name
ORDER BY c.last_name, c.first_name;
```

customer_id	last_name	first_name	max_payment
505	ABNEY	RAFAEL	9.99
504	ADAM	NATHANIEL	9.99
36	ADAMS	KATHLEEN	9.99
96	ALEXANDER	DIANA	7.99
470	ALLARD	GORDON	10.99
27	ALLEN	SHIRLEY	8.99
220	ALVAREZ	CHARLENE	10.99
11	ANDERSON	LISA	9.99
326	ANDREW	JOSE	7.99
183	ANDREWS	IDA	5.99
449	AQUINO	OSCAR	8.99
368	ARCE	HARRY	9.99

Task 3: Duration (days) for each rental, shortest first

```sql
SELECT
  r.rental_id,
  r.rental_date,
  r.return_date,
  DATEDIFF(r.return_date, r.rental_date)
      AS duration_days
FROM rental AS r
WHERE r.return_date IS NOT NULL
ORDER BY duration_days ASC, r.rental_id;
```

or

```sql
SELECT
  r.rental_id,
  r.rental_date,
  r.return_date,
  DATEDIFF(r.return_date, r.rental_date)
      AS duration_days,
  TIMESTAMPDIFF(HOUR, r.rental_date, r.return_date)
      AS duration_hours
FROM rental AS r
WHERE r.return_date IS NOT NULL
ORDER BY duration_hours ASC, duration_days ASC,
r.rental_id
```

rental_id	rental_date	return_date	duration_days	duration_hours
14967	2005-08-22 06:46:03	2005-08-23 01:02:03	1	18
15050	2005-08-22 10:07:52	2005-08-23 04:45:52	1	18
15093	2005-08-22 11:39:03	2005-08-23 06:16:03	1	18
15325	2005-08-22 20:27:38	2005-08-23 15:24:38	1	18
15403	2005-08-22 23:18:10	2005-08-23 18:08:10	1	18
15697	2005-08-23 10:04:36	2005-08-24 05:00:36	1	18
15727	2005-08-23 11:28:49	2005-08-24 06:24:49	1	18
15914	2005-08-23 17:49:26	2005-08-24 12:20:26	1	18
32	2005-05-25 04:06:21	2005-05-25 23:55:21	0	19
157	2005-05-26 01:25:21	2005-05-26 21:17:21	0	19

Task 4: Total Revenue per Store

```
SELECT
  s.store_id,
  SUM(p.amount) AS total_revenue
FROM payment AS p
JOIN staff AS st ON st.staff_id = p.staff_id
JOIN store AS s  ON s.store_id = st.store_id
GROUP BY s.store_id
ORDER BY s.store_id;
```

store_id	total_revenue
1	33482.50
2	33924.06

Task 5: Average Payment Amount per Film

```sql
SELECT
  f.film_id,
  f.title,
  AVG(p.amount) AS average_payment
FROM payment AS p
JOIN rental AS r ON r.rental_id = p.rental_id
JOIN inventory AS i ON i.inventory_id = r.inventory_id
JOIN film AS f ON f.film_id = i.film_id
GROUP BY f.film_id, f.title
ORDER BY f.title;
```

film_id	title	average_payment
1	ACADEMY DINOSAUR	1.598696
2	ACE GOLDFINGER	7.561429
3	ADAPTATION HOLES	3.156667
4	AFFAIR PREJUDICE	3.990000
5	AFRICAN EGG	4.323333
6	AGENT TRUMAN	6.037619
7	AIRPLANE SIERRA	5.523333
8	AIRPORT POLLOCK	5.712222
9	ALABAMA DEVIL	5.990000
10	ALADDIN CALENDAR	5.729130

Task 6: Customers with ≥ 1 unreturned rental (return_date IS NULL)

```
SELECT
  c.customer_id,
  c.last_name,
  c.first_name,
  SUM(r.return_date IS NULL) AS overdue_count
FROM customer AS c
JOIN rental AS r ON r.customer_id = c.customer_id
GROUP BY c.customer_id, c.first_name, c.last_name
HAVING overdue_count > 0
ORDER BY overdue_count DESC, c.last_name, c.first_name;
```

customer_id	last_name	first_name	overdue_count
75	SANDERS	TAMMY	3
560	ARCHULETA	JORDAN	2
60	BAILEY	MILDRED	2
448	BETANCOURT	MIGUEL	2
457	GAVIN	BILL	2
284	GREGORY	SONIA	2
15	HARRIS	HELEN	2

Task 7: Total payment per month (revenue trends)

```
SELECT
  YEAR(p.payment_date) AS year_num,
  MONTHNAME(p.payment_date) AS month_name,
  SUM(p.amount) AS total_revenue
FROM payment AS p
GROUP BY YEAR(p.payment_date), MONTH(p.payment_date), MONTHNAME(p.payment_date)
ORDER BY year_num, MONTH(p.payment_date);
```

payment_id	customer_id	staff_id	rental_id	amount	payment_date	last_update
1	1	1	76	2.99	2005-05-25 11:30:37	2006-02-15 22:12:30
2	1	1	573	0.99	2005-05-28 10:35:23	2006-02-15 22:12:30
3	1	1	1185	5.99	2005-06-15 00:54:12	2006-02-15 22:12:30
4	1	2	1422	0.99	2005-06-15 18:02:53	2006-02-15 22:12:30
5	1	2	1476	9.99	2005-06-15 21:08:46	2006-02-15 22:12:30

year_num	month_name	total_revenue
2005	May	4823.44
2005	June	9629.89
2005	July	28368.91
2005	August	24070.14
2006	February	514.18